GeoPKI

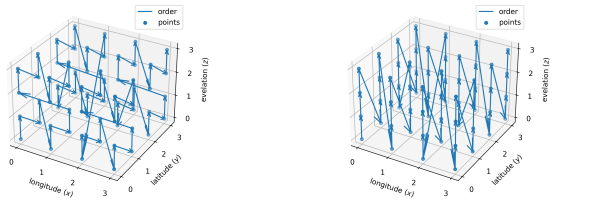
I. Model

The earth ellipsoid model of the existing world geodetic system (WGS) 84 [1] is used. WGS 84 defines the *semi-major axis* (half of the largest diameter of an ellipsoid, at the equator) a to be $6'378'137.0$ meters [1]. The *semi-minor axis* (the polar radius) b is defined as $6'356'752.3142$ meters [1]. Standard polar coordinates (longitude, latitude) are used for referencing points on the planet's two-dimensional surface and a third coordinate, elevation, extends this to three dimensions. Longitude, latitude and elevation together refer to a unique point in three-dimensions.

Each of the three coordinates is discretized in a way that preserves meter-precision in the worst case. With the semi-major axis a, the maximum circumference of the ellipsoid equals $C_x := 2a\pi$ meters. Using base two, $B_x := \lceil \log_2(C_x) \rceil = 26$ bits for the longitude thus suffice to represent each possible meter value in the worst case. For the latitude the circumference is $2b\pi$ meters but we only need to be able to represent half of it, so $C_y := b\pi$. Thus $\lceil \log 2(C_y) \rceil = 25 = B_x - 1$ bits would suffice. Being slightly inefficient, $B_y := B_x =: B_{x,y}$ bits are used to make the binary format a bit easier to deal with. For elevation there is no natural bound so one has to be fixed. At the time of writing, the highest point on earth is below $10'000$ meters above sea level [2] and the lowest layer of earth's atmosphere, the troposphere, extends to about 18 km [3]. The Mariana Trench at about $10'000$ m below sea level [4] is likely a bound not exceeded in the near future. The necessity of allowing volumes below sea level arises from countries such as the Netherlands where parts of the surface's elevation is below sea level [5] . Moreover in the future, below sea level research stations might be built. Thus the range starting at a depth of $D := -10'000$ m up to an elevation of $H := 20'000$ m above sea level should cover any current and near-future needs. Covering this range requires $B_z := \lceil \log_2(H - D) \rceil = 15$ bits.

II. Data Structure

Inspired by F-PKI [6], a sparse merkle hash tree (SMT) is used. Each leaf represents a full discretized coordinate using $B := 2 \cdot B_{x,y} + B_z = 68$ bits resulting in a total of 2^B leaves. To have spatially close points somewhat close together in the SMT, the space-filling Z-order curve is used to sort the three dimensional locations in one dimension [7]. An example with $B_{x,y} = B_z = 2$ is shown in Figure 1a. GeoPKI uses a slightly modified 3D Z-order curve: Rather than using the full 3D Z-order curve, a 2D Z-order curve over the longitude and latitude is used and points with the same longitude and latitude but different elevation are put in sequence. While this leads to spatially close points being further apart in the 1-dimensional sorting, this modification allows for shorter proofs under some assumptions (will be discussed in Section III) and even results in a nicer binary representation. Figure 1b contrasts this modification with the standard 3D Z-order curve shown in Figure 1a.



(a) 3D Z-Order Curve

(b) 2D Z-Order Curve with elevation in sequence

Fig. 1: Difference between a standard 3D Z-order curve and the one used by GeoPKI when using two bits for all coordinates.

A. Leaf Binary Format

Each leaf of the SMT corresponds to a point and the tree leaves are sorted lexicographically by their binary representation in ascending order. A point tuple $p := (x, y, z)$ at longitude x, latitude y and elevation z with

$$(x, y, z) \in [-180, 180] \times [-90, 90] \times [D, H]$$

is first discretized to integer coordinates as follows:

$$\begin{aligned} p' &:= (x_d, y_d, z_d) \\ &= \left(\left\lfloor \frac{x + 180}{360} \cdot C_x \right\rfloor, \left\lfloor \frac{y + 90}{180} \cdot C_y \right\rfloor, \lfloor z - D \rfloor \right) \end{aligned} \tag{1}$$

After this transformation the following folds:

$$p' \in [0, 2^{B_x}) \times [0, 2^{B_y}) \times [0, 2^{B_z})$$

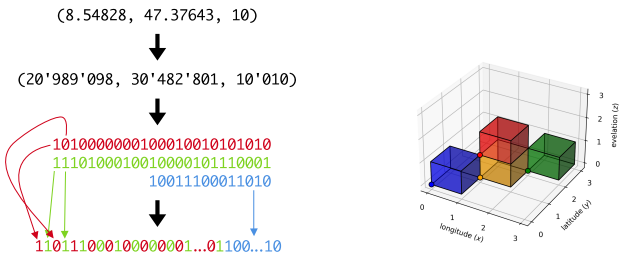
This formula holds since the first term in the longitude and latitude tuples can be at most 1 and since $C_x < 2^{B_{x,y}}$, $C_y < 2^{B_{x,y}}$ and $H - D < 2^{B_z}$ hold. The three resulting integer numbers can now be represented using $B_{x,y}$, $B_{x,y}$ and B_z bits, respectively. Let $p'' := (x_b, y_b, z_b)$ be their big-endian integer binary encoding with

$$p'' \in \{0,1\}^{B_{x,y}} \times \{0,1\}^{B_{x,y}} \times \{0,1\}^{B_z}$$

For $c \in \{x, y, z\}$ and for any integer $i \in \{0, ..., B_c - 1\}$, let $c_{b,i}$ denote the i-th bit of c_b. Moreover let $||$ denote the concatenation operator. The binary representation p_b of the point p is then defined as

$$p_b = \left(\Big|\Big|_{i=0}^{B_{xy}-1} x_{b,i} || y_{b,i} \right) || \left(\Big|\Big|_{i=0}^{B_z-1} z_{b,i} \right)$$

In words, the binary representation of p is the interleaving of the binary representation for the longitude and latitude concatenated to the binary representation of the elevation. Figure 2a shows an example of this binary encoding.



(a) Binary Representation of a (longitude, latitude, elevation) tuple.

(b) Geometric Representation

Fig. 2: The binary and geometric representation of a leaf

B. Leaf Geometric Representation

Due to the floor functions in Equation (1), the point p'' represents the volume where p'' is in the corner with the smallest coordinates in all three dimensions. By replacing the floor functions by mathematically rounding or the ceil function the volumetric representation changes. In case of the floor functions shown in Equation (1), the point corresponding to p'' is

$$\left(x_d \cdot \frac{360}{C_x} - 180, x_d \cdot \frac{180}{C_y} - 90, z_d + D\right) \tag{2}$$

and the volume represented by point p'' is given by

$$\left[x_d \cdot \frac{360}{C_x} - 180, (x_d+1) \cdot \frac{360}{C_x} - 180\right) \times \left[y_d \cdot \frac{180}{C_y} - 90, (y_d+1) \cdot \frac{180}{C_y} - 90\right) \times [z_d + D, z_d + D + 1) \tag{3}$$

Figure 2b shows the points and the corresponding volumes in the same color.

C. Intermediate Nodes

Intermediate nodes in the SMT also correspond to a point and a volume following the same pattern as for the leaf nodes. The binary string corresponding to an intermediate node is simply the longest common prefix of it's two children and covers the whole volume of its two children. The left child is the node with the binary representation equal to the intermediate node one's concatenated with a zero and the right child concatenated with a one. The formula for computing the volume becomes more complex for intermediate nodes as it requires a case distinction: Let s be the binary string of an intermediate node.

Case 1: Assume length of s is exactly $2 \cdot B_{x,y}$. Then the corresponding x_d and y_d are well defined as for the leaves. The corresponding point is

$$\left(x_d \cdot \frac{360}{C_x} - 180, y_d \cdot \frac{180}{C_y} - 90, D\right) \tag{4}$$

and the covered volume is given by

$$\left[x_d \cdot \frac{360}{C_x} - 180, (x_d+1) \cdot \frac{360}{C_x} - 180\right) \times \left[y_d \cdot \frac{180}{C_y} - 90, (y_d+1) \cdot \frac{180}{C_y} - 90\right) \times [D, H) \tag{5}$$

Case 2: Assume length of s is strictly greater than $2 \cdot B_{x,y}$. Then the corresponding x_d and y_d are also well defined as for the leaves. Let s_z be the remaining bits after the first $2 \cdot B_{x,y}$, let l be it's length and let $s_{z,i}$ for $i \in \{0, ..., l-1\}$ be the i-th bit of s_z. The point corresponding to this bit string is

$$\left(x_d \cdot \frac{360}{C_x} - 180, y_d \cdot \frac{180}{C_y} - 90, D + \sum_{i=0}^{l-1} 2^{B_z - 1 - i} \cdot s_{z,i}\right) \tag{6}$$

and the covered volume is given by

$$\left[x_d \cdot \frac{360}{C_x} - 180, (x_d+1) \cdot \frac{360}{C_x} - 180\right) \times \left[y_d \cdot \frac{180}{C_y} - 90, (y_d+1) \cdot \frac{180}{C_y} - 90\right) \times \left[D + \sum_{i=0}^{l-1} 2^{B_z - 1 - i} \cdot s_{z,i}, D + \left(\sum_{i=0}^{l-1} 2^{B_z - 1 - i} \cdot s_{z,i}\right) + 2^{B_z - l}\right) \tag{7}$$

Case 3: Assume length of s is strictly smaller than $2 \cdot B_{x,y}$. In this case there are only partial bits for the longitude and latitude but certainly know that the full z dimension is covered. Let l be the length of s. Let s_x and s_y be the bit prefixes encoded in s corresponding to the longitude and latitude, let l_x and l_y be the lengths of s_x and s_y, and let $s_{x,i}$ and $s_{y,j}$ for $i \in \{0, ..., l_x - 1\}, j \in \{0, ..., l_y - 1\}$ correspond to the i-th and j-th bit of s_x and s_y, respectively. It holds that $l = l_x + l_y$ and it is possible that l_x is by exactly one larger than l_y, otherwise $l_x = l_y$. This directly follows from the interleaved binary encoding described in Section II-A. Let x'_d and y'_d be the most-precise integers that can be re-constructed with the given bits:

$$x'_d := \sum_{i=0}^{l_x - 1} 2^{B_{x,y} - 1 - i} \cdot s_{x,i} \tag{8}$$

$$y'_d := \sum_{i=0}^{l_y - 1} 2^{B_{x,y} - 1 - i} \cdot s_{y,i} \tag{9}$$

Then the corresponding point is given by

$$\left(x'_d \cdot \frac{360}{C_x} - 180, y_d \cdot \frac{180}{C_y} - 90, D\right) \tag{10}$$

and the covered volume by

$$\left[x'_d \cdot \frac{360}{C_x} - 180, (x_d + 2^{B_{x,y} - l_x}) \cdot \frac{360}{C_x} - 180\right) \times \left[y'_d \cdot \frac{180}{C_y} - 90, (y_d + 2^{B_{x,y} - l_y}) \cdot \frac{180}{C_y} - 90\right) \times [D, H) \tag{11}$$

It is worth nothing is that in GeoPKI, intermediate nodes also store a set of certificates. When using a tree representing the geography (which can reduce proof sizes as described in Section IV), this is almost a requirement. Without this optimization, almost no leaf would be empty as almost all pieces of land on earth are claimed by at least one country. This would require enormous storage costs and result in less efficient queries.

D. Sparse Merkle Hash Tree (SMT)

The just described tree is combined with a SMT. As in F-PKI [6], each leaf stores a set of certificates. Let $C_0, C_1, ..., C_{n-1}$ be the sequence of certificates located in a leaf, sorted lexicographically by their SHA-256 hashes in ascending order. The hashes are computed on the PEM encoding of the X.509 certificates. ***cyrill:*** *sort by sha256 hash and then by binary representation? (should never occur though as then we would have bigger problems due to a sha-256 collision in a certificate...)* ***nico:*** *I hope the sentence clearer now? I just wanted to clearly specify what the hash is computed on. It could be the decoded binary value, the base64 encoding, the PEM encoding, the DER encoding, ...* The leaf's hash value is then defined as

$$H(0||H(C_0)||H(C_1)||...||H(C_n)) \tag{12}$$

where H is SHA-256. The hash value of an empty / non-existent leaf in the SMT is $H(0)$. ***cyrill:*** *are empty nodes constructed iteratively starting from the leaf, i.e., H(0) at depth $B_x + B_y + B_z$? Or does an intermediate node with value H(0) at any depth represent an empty subtree?* ***nico:*** *I think F-PKI constructs them iteratively, right? I don't think it matters too much or did you discuss any advantages / disadvantages when deciding that? Using the same value at any depth can save some computations but I have not properly thought about the implications this would have.*

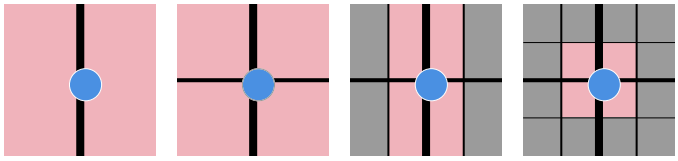
An intermediate node in the tree has two children (two intermediate nodes or two leaves) and in GeoPKI also stores a set of certificates. As before, let $C_0, C_1, ..., C_{n-1}$ be the sequence of certificates located in an intermediate node, sorted lexicographically by the SHA-256 hashes of the PEM encoding in ascending order. Hash h_3 over the list of certificates is defined as

$$h_3 := H(H(C_0)||H(C_1)||...||H(C_n)) \tag{13}$$

Then let h_1 and h_2 be the hash values of the left and right child of the intermediate node, respectively. The hash value of the intermediate node is then defined as

$$\begin{cases} H(1||h_1||h_2) & \text{if } n = 0 \\ H(1||h_1||h_2||h_3) & \text{if } n > 0 \end{cases} \tag{14}$$

The constants 0 and 1 are prefixed in the hash computation to prevent collisions where a given hash $H(a||b)$ for some a and b could simultaneously refer to a leaf with content $a||b$ or a intermediate node with children hash values a and b.



(a) Areas after one split (b) Areas after two splits (c) Areas after three splits (d) Areas after four splits

Fig. 3: The two-dimensional surface (longitude, latitude) after the first, second, third and fourth split. The blue circle shows a possible area to be claimed by a certificate or the desired query area of a client. The area colored in red is the minimum area that has to be retrieved using the given number of splits.

E. Querying the SMT

Since many sizes of volumes can be represented by the binary format described in Sections II-A and II-C, the only type of query a GeoPKI map server supports is such a binary string. Such a binary string has a unique corresponding node in the tree (that might not exist as the tree is sparse). The result then is the set of all certificates stored at intermediate nodes above the specified node (as these represent larger areas covering the queries point) and the set of all certificates stored in the subtree rooted at the specified node as these are within the queried area. The former set corresponds to the set of nodes whose corresponding binary representation are a prefix for the given query. The latter set corresponds to the set of nodes whose corresponding binary representation the given query is a prefix for.

A trivial way to obtain this set of certificates is to walk the tree from the root and collect the certificates stored at the intermediate nodes along the way. If the next bit is a zero, go left, if it is one, go right. Once the node with a binary string corresponding to the query is found, the full subtree rooted at this node is walked recursively until an empty node or a leaf is found and all found certificates are returned.

F. Arbitrary Volumes and Certificate Placement

The discretization used by GeoPKI does not directly fit all requirements. On one hand, claimed volumes will likely never fit exactly one of the voxels. Moreover, clients likely want to query volumes not aligning with the GeoPKI voxels.

In both cases this volume V has to be approximated using the GeoPKI voxels. A trivial approximation the smallest volume encompassing all of V. Unfortunately this results in returning the whole tree if V lies at the intersection of one of the first few division lines. Figure 3 depicts this situation with the blue circle corresponding to V and the red area the minimum area that has to be retrieved at the different levels in the tree. Figure 3a shows the line of the first split. Given the blue circle intersects both area, the above trivial approximation would result in the whole tree being returned for a comparable small area that is just placed unfortunately.

Thus a better approximation is required. The main problem with the trivial approximation is the discrepancy of the area

queried and the returned area: If the client queries for a large area, it is fine to return a lot of data. But in case of a query for a small area, the size of the returned data should be in relation to the queried area. Given the split lines do not move when going to a deeper level in the tree, it is always necessary to return multiple voxels. The number of required voxels is monotonically increasing when going deeper down in the tree but the area covered by them monotonically decreases as the voxel approximation for V becomes more accurate. With decreasing area, the data that has to be returned also monotonically decreases as the subtrees contain at most as much data as any of their super-trees.

1) Claimed Volumes to Voxels: In case of the placement of certificates in the tree, it is infeasible to place certificates claiming a volume V in all leaves covered by V due to the loss in sparsity and the resulting explosion in storage requirements. As noted before, most places on earth are claimed by at least one country and thus the tree would not be sparse at all.

A plausible heuristic balancing the returned data size with the number of subtrees loss in sparsity is to use a subtree level where the volume of V is greater than or equal to a given fraction $f \in [0, 1)$ of the voxel volumes at this level. For instance $f = \frac{1}{4}$ would result in Figure 3c and $f = \frac{1}{2}$ in Figure 3d (assuming all shown areas cover the full elevation spectrum). The effectiveness of this heuristic and a good value for f is to be determined using empirical experiments.

2) Query to Voxels: For the transformation of queries to voxels to be queried the case turns out to be easier. In terms of response size it is always best to query each covered leaf individually. First a set of binary strings is defined to *cover* a volume if and only if the union of the results returned when querying each of them contains all certificates intersecting said volume.

Proof:

Let V be the volume queried by the client and let S be the smallest set of leaf binary strings that covers V. This set exists as the set of all leaves covers any volume that can be queried. The set S is also unique since for each leaf it is possible to individually determine whether it has to be part of S. Next, assume there is some distinct set of binary strings $S' \neq S$ covering V.

Case 1: Assume S' contains only leaf binary strings. Since both cover V and S' is the smallest to do so, S' must be a subset of S. Hence querying S' will return at least as much data as querying S would.

Case 2: Assume S' **not** only contains leaf binary strings. Then S' must contain at least one binary string referring to a non-leaf.

Case 2a: Assume all leaf binary strings descendant of all non-leaf nodes are in S. Then the covered volume is the same and the returned data is exactly the same: Querying a larger area rooted at an intermediate node v returns it's full subtree and thus all certificates stored at its descendant leaves. Similarly, when directly querying all leaves of an intermediate node v amounts to the same full subtree rooted at v if duplicate intermediate nodes are omitted. In both cases the complementary hashes to the root node are required.

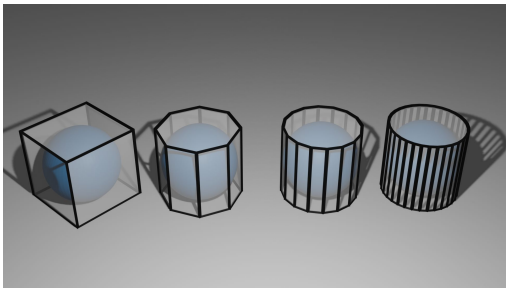


Fig. 4: Approximating a sphere using an extruded polygon with four, eight, sixteen or 32 points.

Case 2b: Assume **not** all leaf binary strings descendant of all non-leaf nodes are in S. Then there is at least one non-leaf node that with a descendant leaf whose binary string is not part of S. The certificates returned when querying S is thus a subset of the certificates returned when querying S'. In case the additional voxels covered by S' are all empty, the exact same set of certificates is returned but the proof of absence of the empty nodes are not required when querying S. In case one of the voxels is non-empty, this is additional data only returned when querying S' but not when querying S. Moreover these non-intersecting subtrees that at least contain one certificate of $\approx$ 2kB size can be replaced by a 32B hashes for the proof when querying S rather than S'.

Since no set S' with a smaller response size can exist, S is optimal.

The question of how to map a point in space and radius to the set of binary strings remains but at least it is now clear what the answer should be.

G. Querying a Sphere

One possibility to query a sphere is to walk the tree and intersect it with the volumes on each level of the tree. This process is repeated recursively until the reached level is a (sparse) "leaf". When walking the tree like this, all information that has to be returned (including the data for the proof) is obtained in one go.

To be more efficient, the first $B_x + B_y$ steps can be performed by intersecting a circle (at the height of the sphere's center where the radius is maximized) with a rectangle (the voxels areas at zero elevation). Analogously for the last B_z steps, the sphere's and voxels' projections to the z or z axis can be intersected. Note that this over-approximates the result since a voxel is counted as intersecting if it intersects the polygon approximation of the circle extruded between its limiting z coordinates. See Figure 4 for a visualization of this with different accuracies for the circle's approximation. Without approximating the circle using a polygon it would correspond to intersecting the voxels with a cylinder. This approach probably works fine for an in-memory data-structure but given the tree's size having it all in memory is infeasible.

An alternative design is to store all non-empty nodes in a data management system such as PostgreSQL and build a spatial index that can be used for querying. A response needs to include all intermediate and leaf nodes whose spatial representation intersects the queried spehere. Non-Empty intersecting nodes are required as they contain certificates relevant for the queried volume but intersecting empty nodes can be omitted as their hash is known or can be computed. Using a spatial index optimized for intersections will likely perform better than a custom implementation. This solutions comes with a new problem: Using exclusively polar coordinates it is not possible to represent a sphere or circle and while PostgreSQL's postgis extension has a `ST_DistanceSphere` function, it cannot make use of an index. Moreover `ST_3DDWithin` does not support polar coordinates and simply treating the polar coordinates as if they were cartesian will cause wrong results. Hence using an actual sphere seems infeasible but it can be over-approximated using a polyhedron which allows the usage of intersections. As in the case of `ST_3DDWithin`, `ST_3DIntersects` also does not support polar coordinates but `ST_Intersects` does. This enables a similar approach as previously described based on two predicates: One predicate ensures the intersection with the polygon approximation of a circle at the sphere's center with the sphere's radius using `ST_Intersects`. The other predicate ensures the z coordinate is within a given sphere's range. As previously noted this over-approximates the result and in will result in more data returned than actually requested. This is due to the two over-approximations: First approximating the circle using a polygon and then second extruding this polygon between the two limiting z coordinates. If `ST_3DDWithin` supported spherical coordinates, no approximation would be required and it would likely result in a more efficient query plan as the approach from before likely involves using the spatial index (R-Tree) to filter based on the first predicate and then perform a table scan for the second.

An alternative approach would be to limit a input sphere's radius to a value where the differences between polar and cartesian coordinates are not significant and thus `ST_3DDWithin` could be used. The risk is that this results in under-approximating the data which would make the server seemingly behave malicious.

Alternatively `ST_3DIntersects` could be used where sphere is approximated using a polyhedron getting rid of one of the two over-approximations. Since `ST_3DIntersects` does not support polar coordinates, it would be required to limit the sphere's maximum radius to mitigate the possibility of under approximating the sphere's actual volume If `ST_3DIntersects` supported geographical coordinates (which it does not), limiting the sphere's radius would no longer be required. The just described approach works for all extruded polygon shapes and thus all shapes that can be approximated using these.

MySQL has equivalent functions for two dimensional data but apparently not for three dimensional data. Given the above, a cartesian coordinate system as proposed in the original GeoPKI [8] paper is easier to query but coordinate transformations are more cumbersome.

H. Proofs of Presence and Absence

Given a query for a volume V, the map server should prove that the returned data is in fact in the SMT and that no certificate was omitted. As described in Section II-E, the server directly only supports queries for binary strings that have a unique corresponding node v in the SMT. Let $\mathcal{C}$ with $C \in \mathcal{C}$ be the set of all certificates stored at v. The server has to send the set $\{H(x) \mid \forall x \in \mathcal{C} \backslash \{C\}\}$ to the client as it is required to compute the node's hash (see Equations 12 and 13). As described in Section II-E, if v is an intermediate node, the query result will include the whole subtree rooted at v. Hence a client has all information to compute the hashes of v's children by themselves. These are required to compute the hash of an intermediate node (see Equation 14).

What is left to complete the proof is the set of complementary hashes along the way to the root and the root hash itself as in a standard SMT. Since in GeoPKI the intermediate nodes along the way to the root also have associated certificates, the set of their hashes is also required but as specified in Section II-E, this information is returned to the client anyway and can thus be computed locally.

In a sparse SMT, proving the absence of any certificate in a subtree rooted at v is the same as proving the presence of an empty node. Thus when a client receives a shorted path up to the root than expected, it already has all information required to proof the node's absence in the tree.

Given the binary SMT has 2^B leaves (see Section II), it's depth is B. A proof of presence for a node v at depth $d \in \{0, ..., B-1\}$ (zero is the root itself) then consists of the d complementary node's hash along the path to the root. The size of these hashes is 256 bits or 32 bytes due to the use of SHA-256. Thus in the worst case a proof for a single leaf consists $(B-1) \cdot 32 \text{ B} = 68 \cdot 32 \text{ B} = 2'176 \text{ B} \approx 2 \text{ kB}$ and is thus in the same order as an average X.509 certificate. The size of such a proof is smaller the sparser the tree is and is always smaller if an intermediate node, i.e., not a leaf, is queried.

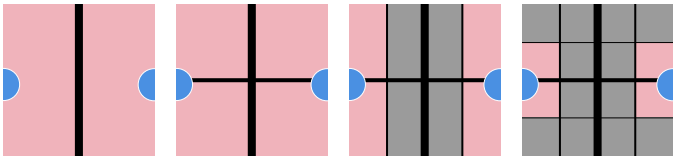
I. Proof of Consistency

As in F-PKI [6] using a consistency tree over the root hashes.

III. Optimizations

First of all it should be noted that the asymmetric design of the binary string with respect to the z dimension allows placing certificates higher in the tree and thus increase sparsity.

Under the assumption most of the world is rather flat with respect to elevation, certificates can often be placed at intermediate nodes covering the full height. This lowers the tree's height and thus reduces the proof sizes. For this to work, clients must not expect the certificates to be placed at a unique node in the tree. The placement does not matter as long as the certificate is stored with *all nodes at one level* whose geometric representation intersect the certificate's claimed volume. This



(a) Areas after one split (b) Areas after two splits (c) Areas after three splits (d) Areas after four splits

Fig. 5: The two-dimensional surface (longitude, latitude) after the first, second, third and fourth split. The blue circle shows a possible area to be claimed by a certificate or the desired query area of a client. The area colored in red is the minimum area that has to be retrieved using the given number of splits.

is because a GeoPKI map server always returns the data along the path to the root together with the whole subtree. Hence when the certificate is placed in the subtree of the queried volume it will always be included but if it is placed higher in the tree it will only be included if the certificate is stored at one of the parent nodes.

Section II-E describes how the map server only serves an answer to a single binary string query. While such an API works, there is room for improvement if correlated queries are answered at the same time. For instance when the client derives the different binary strings to query a single volume as described in Section II-F, the queries share the path to the root from their lowest common ancestor. By replying with a single answer for such a set of queries, the data along this shared path has to be returned just once. Given the use of the Z-order curve (see Section II) to order the leaves, points spatially close should also be somewhat close in the tree and thus have a common ancestor on lower levels. Though in the worst case it is still possible that the only common ancestor is the root even if the two areas are spatially close. This happens when the volume covers both sides of the first split done in the binary tree. This scenario is depicted in Figure 3a. It seems impossible to avoid such a worst-case scenario entirely when using a tree as described in Section IV.

Considering that earth is a sphere and that the coordinates wrap around, the situation becomes even more difficult since a similar situation happens at the points where the coordinate system wraps around. This scenario is depicted in Figure 5. This wrap-around can be interpreted as being part of the first split since the problem occurs exactly on the opposite side of the first split when considering the x-axis as wrapping around. To reduce it's impact it would be possible to shift the longitude coordinate system west by $30°$. This way this circle around earth passes through the Atlantic Ocean and Pacific Ocean rather than central Europe.

An additional optimization would be to set B_y to 25 rather than 26 bits. This slightly complicates the notations, definitions and formats but is otherwise straight-forward to implement.

IV. Alternative Designs

An alternative design could use a (hybrid-)quadtree or (hybrid-)octree where most intermediate nodes have four or eight children, respectively. "Hybrid" and "most" because $B_x = B_y > B_z$ which means the volume cannot be subdivided fully using a quad- or octree. Hence when using an octree with eight children, the z dimension is already exhausted after B_z splits and will from there continue to only have four children (i.e. a quadtree). When using a quadtree, the first $B_{x,y}$ splits would solely concern the x and y dimensions and after that the children will be binary trees of height B_z only splitting the z dimension. This will decrease the tree's height but increase the proof size due to the larger number of neighboring hashes that have to be provided on each level. For the hybrid-quadtree, the tree height would be $\log_4(2^{2\cdot B_{x,y}}) = \log_4(4^{B_{x,y}}) = B_{x,y} = 26$ plus the $\log_2(2^{B_z}) = B_z = 15$ for the binary subtrees. For the hybrid-octree the tree height would be $\log_8(2^{B_z}) = \log_8(8^{B_z/3}) = B_z/3 = 5$ plus the $\log_4(2^{B-B_z}) = \log_4(2^{2\cdot B_{x,y}}) = B_{x,y} = 26$ for the quad-subtrees. The proof size of a leaf would thus increase to $(26\cdot(4-1)+(15-1)\cdot(2-1))\cdot 32$ B $= 2'944$ B ≈ 3 kB and $(5\cdot(8-1)+(26-1)\cdot(4-1))\cdot 32$ B $= 3'520$ B ≈ 3.5 kB, respectively. The ones are subtracted from the tree height since the on the last level no complementary hash has to be returned. The ones subtracted from the number of neighbors is due to the fact that the hash resulting from the leaf itself does not have to be provided as it can be computed by the client.

Unfortunately the worst case scenario remains: Two spatially adjacent volumes intersecting the lines of the first split only share the root node as a common ancestor. When using a tree this will always be the case due to nodes having a single parent. Consider single dimensional discrete consecutive numbers and a binary tree. Each number (except the start and end) has two numerically neighbors but can only have one neighbor in the tree. This fact also holds for group of leaves and thus there will always be two numerically close neighbors just sharing the root node as their ancestor.

To maintain consistency with the already introduced binary representation of volumes, one can consider the quad-tree part (i.e. the first $B_{x,y}$ levels) to add two bits to the prefix on each level rather than just one. The last B_z levels would then each just add a single bit as before.

Since it is not yet clear if and to what extent the usage a more complex tree structure could be beneficial and hence the more simple binary tree is preferred for the moment.

V. Caveats

Since longitude (x) and latitude (y) coordinates that are split in half numerically represent polar coordinates on a sphere, numerically equidistant points, areas or volumes are not equidistant on earth. Near the poles the longitudinal circumference is smaller but represented using the same numerically range resulting in higher precision. Hence the discretization range was chosen based on the semi-major axis at the equator where the achieved precision is smallest.

VI. Certificate Format

The claimed volume has to be specified in the X.509 certificate. The used format should be flexible enough to

support a variety of use cases such as a set of disjoint volumes, rectangular but also circular volumes. Open Street Maps uses polygons and RFC 7946 specifies the "geospatial data interchange format" GeoJSON [9] that also makes use of polygons. One option would be to define additional types allowing for three dimensional polygons, another to use the "properties" field defined in RFC 7946. When going with the latter approach, each certificate could define a `GeometryCollection` of `Polygon`s or `MultiPolygon`s, each with an `elevation_from` and `elevation_to` property indicating the inclusive minimum and maximum elevation, respectively. Since the GeoJSON uses a text-encoding, serializing the data using e.g. protocol buffers as done by Mapbox [10] will result in a smaller encoding and fast read and write operations.

REFERENCES

[1] NGA geomatics - WGS 84. [Online]. Available: https://earth-info.nga.mil/?dir=wgs84&action=wgs84

[2] N. O. a. A. A. US Department of Commerce. What is the highest point on earth as measured from earth's center? [Online]. Available: https://oceanservice.noaa.gov/facts/highestpoint.html

[3] S. K. Dhaka and V. Kumar, "Chapter 1 - composition and thermal structure of the earth's atmosphere," in *Atmospheric Remote Sensing*, ser. Earth Observation, A. Kumar Singh and S. Tiwari, Eds. Elsevier, pp. 1–18. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B9780323992626000237

[4] J. V. Gardner, A. A. Armstrong, B. R. Calder, and J. Beaudoin, "So, how deep is the mariana trench?" vol. 37, no. 1, pp. 1–13, publisher: Taylor & Francis _eprint: https://doi.org/10.1080/01490419.2013.837849. [Online]. Available: https://doi.org/10.1080/01490419.2013.837849

[5] P. Kabat, W. van Vierssen, J. Veraart, P. Vellinga, and J. Aerts, "Climate proofing the netherlands," vol. 438, no. 7066, pp. 283–284, number: 7066 Publisher: Nature Publishing Group. [Online]. Available: https://www.nature.com/articles/438283a

[6] L. Chuat, C. Krähenbühl, P. Mittal, and A. Perrig, "F-PKI: Enabling innovation and trust flexibility in the HTTPS public-key infrastructure," in *Proceedings 2022 Network and Distributed System Security Symposium*. [Online]. Available: http://arxiv.org/abs/2108.08581

[7] G. M. Morton, "A computer oriented geodetic data base; and a new technique in file sequencing."

[8] T. H.-J. Kim, V. Gligor, and A. Perrig, "GeoPKI: Converting spatial trust into certificate trust," in *Public Key Infrastructures, Services and Applications*, ser. Lecture Notes in Computer Science, S. De Capitani di Vimercati and C. Mitchell, Eds. Springer, pp. 128–144.

[9] H. Butler, M. Daly, A. Doyle, S. Gillies, T. Schaub, and S. Hagen, "The GeoJSON format," num Pages: 28. [Online]. Available: https://datatracker.ietf.org/doc/rfc7946

[10] Mapbox, "Geobuf," original-date: 2014-07-14T20:27:58Z. [Online]. Available: https://github.com/mapbox/geobuf